SEMINÁRIO AV2 · GRAFOS

# Componentes Conexos

Relatório: conceito, implementação em Java e aplicação ao isolamento de municípios por uma cheia

**Equipe**
Pedro Felipe
Leonardo Costa
Pedro Melo

**Linguagem**
Java 17 (sem bibliotecas externas)

Outubro de 2026

# Sumário


**1. Introdução** 3

**2. Conceito** 3

2.1 Definições . . . 3
2.2 O problema computacional . . . 4
2.3 Pseudocódigo: busca em profundidade (DFS) e em largura (BFS) . . . 4
2.4 Pseudocódigo: União-Busca . . . 5
2.5 Correção . . . 6
2.6 Complexidade . . . 7
2.7 Grafos direcionados . . . 7

**3. Implementação** 7

3.1 Linguagem e organização . . . 7
3.2 Representação do grafo: listas de adjacência . . . 8
3.3 Busca em profundidade iterativa . . . 9
3.4 Busca em largura . . . 10
3.5 União-Busca . . . 10
3.6 Um resultado único para os três algoritmos . . . 10
3.7 Testes . . . 11
3.8 Desempenho medido . . . 11

**4. Aplicação: municípios isolados por uma cheia** 12

4.1 O problema . . . 12
4.2 Modelagem . . . 13
4.3 A rede em condições normais . . . 13
4.4 Após a cheia . . . 14
4.5 Consultas de conectividade . . . 15
4.6 Plano de reparo: União-Busca incremental . . . 15
4.7 Ordem de reparo sugerida . . . 16
4.8 Limitações e extensões . . . 17

**5. Conclusão** 17

**6. Referências** 18

**Apêndice A — Como executar** 19

**Apêndice B — Saída completa da aplicação** 20

# 1. Introdução

Muitas perguntas práticas sobre redes se resumem a uma só: *quem consegue alcançar quem?* Quais computadores de uma rede ainda se comunicam depois que um cabo é cortado, quais usuários de uma rede social formam grupos sem contato entre si, quais pixels de uma imagem pertencem ao mesmo objeto, quais cidades continuam ligadas por estrada depois de uma enchente. Em todos esses casos, a resposta é dada pelos **componentes conexos** do grafo que modela a rede.

Este relatório apresenta o tema em três partes, como pede o enunciado do seminário:

- **Conceito** (Seção 2): a definição formal do problema, as propriedades que garantem que ele está bem definido, o pseudocódigo de três algoritmos — busca em profundidade (DFS), busca em largura (BFS) e União-Busca —, a prova de correção e a análise de complexidade;
- **Implementação** (Seção 3): as decisões tomadas na implementação em Java, com a justificativa de cada uma, os trechos relevantes do código explicados, os testes e as medições de desempenho;
- **Aplicação** (Seção 4): a modelagem, como grafo, da rede de estradas de 18 municípios do Litoral Leste e do Baixo Jaguaribe (Ceará) e o uso dos componentes conexos para responder às perguntas que uma cheia impõe à Defesa Civil: quais municípios ficaram isolados, quais grupos estão sem apoio e em que ordem reabrir as estradas.

Todos os números, tabelas e trechos de código deste documento foram extraídos diretamente do programa e da sua execução; o Apêndice A explica como reproduzi-los.

# 2. Conceito

## 2.1 Definições

**Definição 1 (grafo).** Um grafo não direcionado é um par $G = (V, E)$, em que $V$ é um conjunto finito de **vértices** e $E \subseteq \{\{u, v\} : u, v \in V\}$ é um conjunto de **arestas**. Escrevemos $n = |V|$ e $m = |E|$. A vizinhança de $u$ é $\mathrm{Adj}[u] = \{v : \{u, v\} \in E\}$.

**Definição 2 (caminho e alcançabilidade).** Um caminho de $u$ a $v$ é uma sequência de vértices $v_0 = u, v_1, \ldots, v_l = v$ tal que $\{v_{i-1}, v_i\} \in E$ para todo $1 \le i \le l$ (o caminho de comprimento $l = 0$ é permitido). Dizemos que $v$ é **alcançável** a partir de $u$, e escrevemos $u \sim v$, se existe um caminho de $u$ a $v$.

**Proposição 1.** A relação $\sim$ é uma relação de equivalência em $V$.

*Prova.* É **reflexiva**, pois o caminho de comprimento 0 liga $u$ a si mesmo; é **simétrica**, pois, como as arestas não têm orientação, um caminho de $u$ a $v$ percorrido ao contrário é um caminho de $v$ a $u$; e é **transitiva**, pois a concatenação de um caminho de $u$ a $v$ com um de $v$ a $w$ é um passeio de $u$ a $w$, do qual se extrai um caminho eliminando os ciclos. ∎

**Definição 3 (componente conexo).** As classes de equivalência de $\sim$ formam uma partição $V = V_1 \cup \ldots \cup V_k$. Cada subgrafo induzido $G[V_i]$ é um **componente conexo** de $G$; equivalentemente, um componente conexo é um subgrafo conexo **maximal** (não pode ser aumentado sem perder a conexidade). O número de componentes é denotado $c(G) = k$, e $G$ é **conexo** se e somente se $c(G) = 1$.

A Proposição 1 é o que torna o problema bem definido: por ser uma relação de equivalência, a alcançabilidade separa os vértices em grupos disjuntos, e todo vértice pertence a exatamente um componente — inclusive um vértice isolado, que forma sozinho um componente. Duas consequências serão usadas adiante:

- cada componente com $n_i$ vértices precisa de pelo menos $n_i - 1$ arestas (uma árvore geradora); logo $m \geq n - c(G)$;
- acrescentar uma aresta $\{u, v\}$ diminui $c(G)$ em exatamente 1 se $u$ e $v$ estavam em componentes diferentes, e não altera $c(G)$ caso contrário. Por isso, para reunir $k$ componentes em um só bastam — e são necessárias — exatamente $k - 1$ arestas escolhidas entre componentes distintos.

## 2.2 O problema computacional

**Problema (componentes conexos).**
**Entrada:** um grafo não direcionado $G = (V, E)$.
**Saída:** o número $k = c(G)$ e uma função de rótulos $c : V \to \{1, \ldots, k\}$ tal que, para todos $u, v \in V$, $c(u) = c(v) \Leftrightarrow u \sim v$.

Com os rótulos calculados uma única vez, a pergunta "$u$ e $v$ estão conectados?" passa a ser respondida em tempo constante, comparando $c(u)$ com $c(v)$, sem precisar procurar um caminho a cada consulta.

**Exemplo.** O grafo da Figura 1 tem $n = 10$ vértices e $m = 7$ arestas — {A,B}, {A,C}, {B,D}, {C,D}, {E,F}, {F,G} e {H,I} — e $c(G) = 4$ componentes: {A, B, C, D}, {E, F, G}, {H, I} e {J}. O vértice J, sem arestas, é um componente por si só.

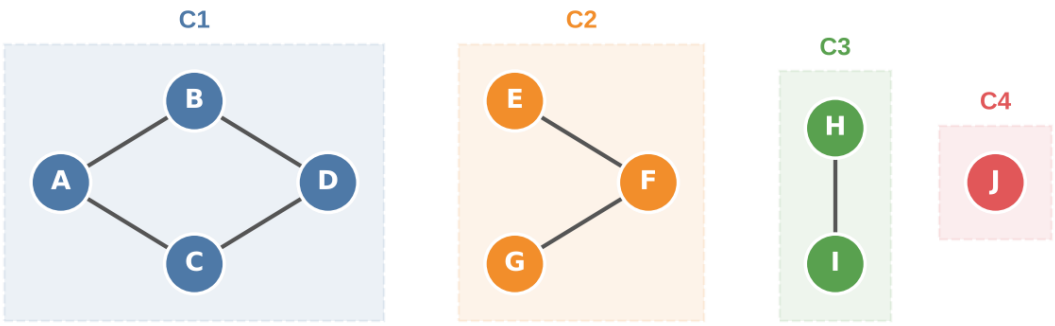



*Figura 1 — Grafo de exemplo com 4 componentes conexos (cada cor é um componente).*

Note que a aresta {C, D} fecha um ciclo dentro de C1: ela não muda a conectividade. É exatamente essa aresta que a União-Busca reconhece como "já no mesmo conjunto" (Seção 2.4).

## 2.3 Pseudocódigo: busca em profundidade (DFS) e em largura (BFS)

A ideia é simples: percorrem-se os vértices em qualquer ordem e, a cada vértice $s$ ainda sem rótulo, inicia-se uma busca que rotula com um novo número $k$ tudo o que for alcançável a partir de $s$. Cada busca descobre um componente inteiro.

```
COMPONENTES-DFS(G)
para cada v ∈ V faça rótulo[v] ← NIL
k ← 0
para cada s ∈ V faça
    se rótulo[s] = NIL então
        k ← k + 1
        EXPLORAR(G, s, k)
devolva (k, rótulo)

EXPLORAR(G, s, k)
rótulo[s] ← k
P ← pilha vazia;  EMPILHAR(P, s)
enquanto P não está vazia faça
    u ← DESEMPILHAR(P)
    para cada w ∈ Adj[u] faça
        se rótulo[w] = NIL então
            rótulo[w] ← k            ▷ marca ao empilhar: cada vértice entra uma vez
            EMPILHAR(P, w)
```

*Algoritmo 1 — Componentes conexos por busca em profundidade (versão iterativa).*

A **BFS** é idêntica, trocando a pilha *P* por uma fila: EMPILHAR vira ENFILEIRAR (no fim) e DESEMPILHAR vira DESENFILEIRAR (do início). A ordem de descoberta muda — a BFS visita os vértices em camadas de distância crescente a partir de *s* —, mas o conjunto rotulado por cada busca é o mesmo, porque ele depende apenas de quem é alcançável, não da ordem da visita.

O traço abaixo é a saída real do programa (comando `exemplo`) executando o Algoritmo 1 no grafo da Figura 1:

```
DFS iterativa, passo a passo:
  componente 1: começa em A
    desempilha A -> descobre: B, C   pilha = [B C]
    desempilha C -> descobre: D      pilha = [B D]
    desempilha D -> descobre: -      pilha = [B]
    desempilha B -> descobre: -      pilha = []
  componente 2: começa em E
    desempilha E -> descobre: F      pilha = [F]
    desempilha F -> descobre: G      pilha = [G]
    desempilha G -> descobre: -      pilha = []
  componente 3: começa em H
    desempilha H -> descobre: I      pilha = [I]
    desempilha I -> descobre: -      pilha = []
  componente 4: começa em J
    desempilha J -> descobre: -      pilha = []
```

## 2.4 Pseudocódigo: União-Busca

A União-Busca (*Union-Find* ou *Disjoint Set Union*) resolve o problema sem percorrer o grafo: começa com cada vértice em um conjunto unitário e, para cada aresta, une os conjuntos das duas pontas. Cada conjunto é guardado como uma árvore cujo representante é a raiz.

```
COMPONENTES-UNIÃO-BUSCA(G)
para cada v ∈ V faça pai[v] ← v;  tam[v] ← 1
k ← |V|
para cada {u, v} ∈ E faça
    se UNIR(u, v) então k ← k − 1
devolva (k, BUSCAR(v) para cada v ∈ V)

BUSCAR(x)
enquanto pai[x] ≠ x faça
    pai[x] ← pai[pai[x]]            ▷ compressão de caminho ("halving")
    x ← pai[x]
devolva x

UNIR(a, b)
ra ← BUSCAR(a);  rb ← BUSCAR(b)
se ra = rb então devolva FALSO       ▷ já estavam no mesmo componente
se tam[ra] < tam[rb] então troque ra ↔ rb
pai[rb] ← ra;  tam[ra] ← tam[ra] + tam[rb]   ▷ união por tamanho
devolva VERDADEIRO
```

*Algoritmo 2 — Componentes conexos por União-Busca.*

No exemplo, as arestas {A,B}, {A,C}, {B,D}, {E,F}, {F,G} e {H,I} unem conjuntos distintos (cada uma reduz $k$ em 1, de 10 até 4), enquanto {C,D} encontra C e D no mesmo conjunto e é descartada, como mostra a saída do programa:

```
União-Busca, aresta por aresta (começa com 10 conjuntos unitários):
  unir(A, B): conjuntos distintos -> une   componentes = 9
  unir(A, C): conjuntos distintos -> une   componentes = 8
  unir(B, D): conjuntos distintos -> une   componentes = 7
  unir(C, D): já no mesmo conjunto -> nada componentes = 7
  unir(E, F): conjuntos distintos -> une   componentes = 6
  unir(F, G): conjuntos distintos -> une   componentes = 5
  unir(H, I): conjuntos distintos -> une   componentes = 4
```

A Figura 2 mostra a floresta resultante: uma árvore por componente.

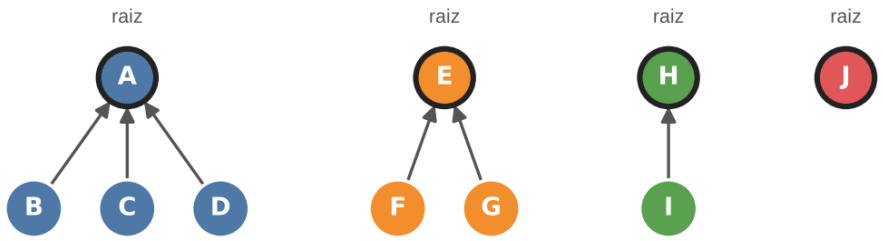



*Figura 2 — Floresta da União-Busca ao final do exemplo; cada seta aponta para o pai e a raiz é o representante do componente.*

## 2.5 Correção

**Teorema 1.** Ao final de EXPLORAR($G$, $s$, $k$), os vértices com rótulo $k$ são exatamente os vértices alcançáveis a partir de $s$.

*Prova.* ($\subseteq$) Um vértice só recebe o rótulo $k$ ao ser descoberto como vizinho de um vértice $u$ já rotulado com $k$; por indução no instante de rotulação, existe um caminho de $s$ até ele. ($\supseteq$) Suponha que algum vértice alcançável a partir de $s$ termine sem rótulo $k$. Tome um caminho de $s$ até ele e seja $w$ o primeiro vértice do caminho sem rótulo $k$, com predecessor $x$ rotulado. Todo vértice rotulado é empilhado, e o laço só termina com a pilha vazia; logo $x$ foi desempilhado e todos os seus vizinhos foram examinados — inclusive $w$, que teria sido rotulado. Contradição. ∎

Como cada chamada de EXPLORAR rotula exatamente um componente inteiro, e o laço principal só inicia uma busca em vértices ainda sem rótulo, cada componente é encontrado uma única vez: o algoritmo

devolve $k = c(G)$ e rótulos que satisfazem a especificação. A mesma prova vale para a BFS.

Para a União-Busca, vale o **invariante**: depois de processadas as arestas $e_1, \ldots, e_i$, os conjuntos coincidem com os componentes do grafo $(V, \{e_1, \ldots, e_i\})$. Ele vale no início (sem arestas, cada vértice é um componente) e é preservado por UNIR, que funde dois componentes exatamente quando a nova aresta liga vértices de componentes diferentes. Ao final ($i = m$), os conjuntos são os componentes de $G$.

### 2.6 Complexidade

Na DFS e na BFS, cada vértice é rotulado e empilhado (ou enfileirado) **uma única vez**, pois a marcação acontece no momento da inserção. Assim, a lista de adjacência de cada vértice é percorrida uma vez, e o custo total é $O(n + \Sigma_v \text{ grau}(v)) = O(n + 2m) = O(n + m)$ — linear no tamanho do grafo, o melhor possível, já que toda aresta precisa ser examinada ao menos uma vez.

Na União-Busca, com união por tamanho e compressão de caminho, uma sequência de operações custa $O(\alpha(n))$ amortizado por operação (Tarjan, 1975), em que $\alpha$ é a inversa da função de Ackermann: $\alpha(n) \leq 4$ para qualquer $n$ que caiba no universo observável. Na prática, é tempo constante.

| Algoritmo | Tempo | Memória extra | Nova aresta depois do cálculo | Consulta “u ~ v?” |
|---|---|---|---|---|
| DFS | O(n + m) | O(n) (pilha + rótulos) | recalcular: O(n + m) | O(1) |
| BFS | O(n + m) | O(n) (fila + rótulos) | recalcular: O(n + m) | O(1) |
| União-Busca | O(n + m·α(n)) | O(n) (pai + tamanho) | UNIR: O(α(n)) amortizado | O(α(n)) |

*Tabela 1 — Comparação dos três algoritmos (n = \|V\|, m = \|E\|).*

A diferença prática está na última coluna: DFS e BFS são **estáticas** — se o grafo ganhar uma aresta, é preciso refazer o cálculo —, enquanto a União-Busca é **incremental** e absorve novas arestas quase de graça. Nenhuma das três trata **remoções** de forma incremental (esse é o problema mais difícil da conectividade dinâmica); na aplicação, por isso, a cheia (que remove estradas) é tratada com DFS, e a reabertura das estradas (que as acrescenta) com União-Busca.

### 2.7 Grafos direcionados

Em grafos direcionados a alcançabilidade deixa de ser simétrica, e o conceito se divide em dois. Os componentes **fracamente conexos** são os componentes do grafo obtido ignorando a orientação das arestas — os mesmos algoritmos deste relatório os resolvem. Os componentes **fortemente conexos** exigem que haja caminho nos dois sentidos entre cada par de vértices e são obtidos, também em $O(n + m)$, pelos algoritmos de Kosaraju ou de Tarjan. Como a rede de estradas da aplicação é de mão dupla, este trabalho trata o caso não direcionado.

## 3. Implementação

### 3.1 Linguagem e organização

Entre as linguagens permitidas (Java ou Python), escolhemos **Java**. A tipagem estática e os vetores de tipos primitivos (`int[]`) deixam explícitas as estruturas do pseudocódigo — rótulos, pilha, fila, vetor de pais — e o desempenho permite medir os algoritmos em grafos com milhões de arestas. O código usa apenas a biblioteca padrão e compila em qualquer JDK a partir da versão 17.

Os três algoritmos implementam a mesma interface, `AlgoritmoComponentes`, e devolvem o mesmo tipo de resultado, `Componentes`. Com isso, o restante do programa (a aplicação, os testes e as medições) trata os algoritmos de forma intercambiável, e compará-los é trivial.

| Classe | Responsabilidade |
|---|---|
| Grafo | Grafo não direcionado com listas de adjacência; vértices identificados por índice e por nome. |
| AlgoritmoComponentes | Interface comum: nome() e calcular(Grafo) → Componentes. |
| BuscaEmProfundidade | DFS iterativa (e a recursiva, mantida para comparação). |
| BuscaEmLargura | BFS com fila em vetor. |
| UniaoBusca | União-Busca com união por tamanho e compressão de caminho; também usada de forma incremental. |
| Componentes | Resultado: rótulo por vértice, quantidade, tamanhos, listagem e consulta conectados(u, v). |
| LeitorGrafo | Leitura do grafo a partir de arquivo texto (UTF-8). |
| AplicacaoEnchente | A aplicação da Seção 4. |
| ExemploDidatico, Desempenho, Main | Exemplo passo a passo, medições de tempo e ponto de entrada. |
| TestesComponentes | 4.073 verificações automatizadas. |

*Tabela 2 — Classes do projeto (pacote componentes).*

## 3.2 Representação do grafo: listas de adjacência

O grafo é guardado como **listas de adjacência**: para cada vértice, a lista dos seus vizinhos. A alternativa, a matriz de adjacência, gasta $\Theta(n^2)$ de memória e faz a busca examinar $n$ posições por vértice, levando o algoritmo a $\Theta(n^2)$. Para um grafo com $10^6$ vértices, a matriz teria $10^{12}$ posições, o que é inviável; as listas ocupam $O(n + m)$ e permitem a busca em tempo linear.

Internamente os vértices são inteiros de 0 a $n - 1$, para que os algoritmos usem vetores simples; um `HashMap` traduz os nomes (por exemplo, “Limoeiro do Norte”) para índices. Um conjunto de chaves impede arestas repetidas, que não alteram a conectividade e só aumentariam o trabalho:

```
1  public boolean adicionarAresta(int u, int v) {
2      validar(u);
3      validar(v);
4      if (!chavesArestas.add(chave(u, v))) return false;
5      adjacencia.get(u).add(v);
6      if (u != v) adjacencia.get(v).add(u);   // laço: guardado uma única vez
7      numArestas++;
8      return true;
9  }
10
11 private static long chave(int u, int v) {
12     int a = Math.min(u, v), b = Math.max(u, v);
13     return ((long) a << 32) | (b & 0xffffffffL);
14 }
```

A aresta $\{u, v\}$ é codificada em um único `long` com o menor índice nos 32 bits altos e o maior nos 32 bits baixos (linhas 12–13), de modo que $\{u, v\}$ e $\{v, u\}$ geram a mesma chave, e a linha 4 descarta a aresta se ela já existir. Como o grafo é não direcionado, cada aresta aparece nas duas listas (linhas 5–6); um laço $\{v, v\}$ é guardado uma vez só.

### 3.3 Busca em profundidade iterativa

```
1  public Componentes calcular(Grafo g) {
2      int n = g.numVertices();
3      int[] rotulo = new int[n];
4      Arrays.fill(rotulo, -1);                // -1 = ainda não visitado
5      int k = 0;
6      int[] pilha = new int[n];               // cada vértice entra na pilha no máximo uma vez
7      for (int s = 0; s < n; s++) {
8          if (rotulo[s] != -1) continue;      // s já pertence a um componente
9          if (recursiva) visitar(g, s, k, rotulo);
10         else explorar(g, s, k, rotulo, pilha);
11         k++;                                // terminou um componente
12     }
13     return new Componentes(rotulo, k);
14 }
15
16 private static void explorar(Grafo g, int s, int k, int[] rotulo, int[] pilha) {
17     int topo = 0;
18     rotulo[s] = k;                          // marca ao empilhar: nunca empilha duas vezes
19     pilha[topo++] = s;
20     while (topo > 0) {
21         int u = pilha[--topo];
22         for (int w : g.vizinhos(u)) {
23             if (rotulo[w] == -1) {
24                 rotulo[w] = k;
25                 pilha[topo++] = w;
26             }
27         }
28     }
29 }
```

O método `calcular` é o laço principal do Algoritmo 1: o vetor `rotulo` começa com −1 (NIL) e cada vértice ainda sem rótulo inicia uma nova exploração (linhas 7–11). Em `explorar`, a pilha é um simples `int[]` de tamanho $n$ com um índice `topo`: como o vértice é marcado **no momento em que é empilhado** (linhas 18 e 24), nenhum vértice entra duas vezes e o vetor nunca transborda. Marcar só ao desempilhar também daria a resposta certa, mas permitiria empilhar o mesmo vértice várias vezes, gastando até $O(m)$ de memória.

**Por que iterativa?** A DFS recursiva é mais curta e é a forma clássica dos livros — e também está no projeto, para comparação. Mas a profundidade da recursão é igual ao comprimento do caminho explorado, e a pilha de chamadas da JVM é limitada (da ordem de 1 MB por *thread*, por padrão; opção `-Xss`). Num grafo que é um caminho, a versão recursiva falha a partir de alguns milhares de vértices:

| Vértices do caminho | DFS recursiva | DFS iterativa |
|---|---|---|
| 1.000 | ok (k = 1) | ok (k = 1) |
| 10.000 | StackOverflowError | ok (k = 1) |
| 100.000 | StackOverflowError | ok (k = 1) |
| 1.000.000 | StackOverflowError | ok (k = 1) |

*Tabela 3 — Execução real (comando desempenho): a recursão estoura a pilha da JVM; a pilha explícita não tem esse limite.*

Com a pilha explícita, o limite passa a ser apenas a memória do *heap*, e o mesmo código atende tanto o exemplo de 10 vértices quanto grafos com milhões deles.

### 3.4 Busca em largura

```
for (int s = 0; s < n; s++) {
    if (rotulo[s] != -1) continue;
    int inicio = 0, fim = 0;
    rotulo[s] = k;
    fila[fim++] = s;
    while (inicio < fim) {
        int u = fila[inicio++];          // retira da frente da fila
        for (int w : g.vizinhos(u)) {
            if (rotulo[w] == -1) {
                rotulo[w] = k;
                fila[fim++] = w;         // insere no fim da fila
            }
        }
    }
    k++;
}
```

A fila também é um vetor de tamanho *n* com dois índices, `inicio` e `fim`. Como cada vértice entra na fila no máximo uma vez, não é preciso fila circular nem `ArrayDeque`: inserir é `fila[fim++] = w` (linha 11) e retirar é `fila[inicio++]` (linha 7), ambos em $O(1)$ e sem criar objetos.

### 3.5 União-Busca

```
public int buscar(int x) {
    while (pai[x] != x) {
        pai[x] = pai[pai[x]];   // compressão de caminho (halving)
        x = pai[x];
    }
    return x;
}

public boolean unir(int a, int b) {
    int ra = buscar(a), rb = buscar(b);
    if (ra == rb) return false;
    if (tam[ra] < tam[rb]) { int t = ra; ra = rb; rb = t; }
    pai[rb] = ra;               // a árvore menor fica embaixo da maior
    tam[ra] += tam[rb];
    componentes--;
    return true;
}
```

As duas otimizações do Algoritmo 2 aparecem em poucas linhas. A **compressão de caminho** (linha 3) usa a variante *halving*: durante a subida até a raiz, cada nó passa a apontar para o avô. Ela é iterativa (não há recursão para estourar) e, ao longo das buscas, achata as árvores. A **união por tamanho** (linhas 12–14) pendura sempre a árvore menor sob a maior, o que limita a altura a $\log_2 n$ mesmo sem compressão. O valor devolvido por `unir` diz se a aresta reduziu o número de componentes — informação que a aplicação usa para separar estradas úteis de estradas redundantes.

### 3.6 Um resultado único para os três algoritmos

A DFS e a BFS numeram os componentes na ordem em que os encontram; a União-Busca identifica cada componente pela raiz da sua árvore, que é um vértice arbitrário. Para que os três devolvam **exatamente o mesmo** vetor de rótulos — e possam ser comparados diretamente nos testes —, os representantes são renumerados na ordem do menor vértice de cada componente:

```java
static Componentes normalizar(int[] representante) {
    int n = representante.length;
    int[] novo = new int[n];
    int[] mapa = new int[n];
    java.util.Arrays.fill(mapa, -1);
    int k = 0;
    for (int v = 0; v < n; v++) {
        int r = representante[v];
        if (mapa[r] == -1) mapa[r] = k++;
        novo[v] = mapa[r];
    }
    return new Componentes(novo, k);
}
```

A renumeração custa $O(n)$ e torna o resultado **canônico**: o componente 1 é sempre o que contém o vértice 0, o componente 2 é o que contém o menor vértice fora do componente 1, e assim por diante.

### 3.7 Testes

O programa `TestesComponentes` executa **4.073 verificações**, todas aprovadas. Além do exemplo da Figura 1 e de casos-limite (grafo vazio, só vértices isolados, caminho, grafo completo, arestas repetidas e laços), o teste principal gera **500 grafos aleatórios** e compara cada algoritmo com uma resposta obtida por um método totalmente independente: o **fecho transitivo** calculado pelo algoritmo de Floyd–Warshall booleano, que decide $u \sim v$ para todos os pares em $O(n^3)$. Também se verifica que a União-Busca incremental acompanha a DFS aresta a aresta, que a DFS recursiva estoura a pilha onde a iterativa não estoura, e que o arquivo da aplicação é lido corretamente.

### 3.8 Desempenho medido

O comando `desempenho` mede o tempo mediano de sete execuções de cada algoritmo, depois de três execuções de aquecimento da JVM, em grafos aleatórios com $n$ de $10^4$ a $10^6$ vértices e $m = 0{,}5n$, $n$ e $2n$ arestas (semente fixa, para que os grafos sejam reproduzíveis).

| V | E | componentes k | DFS (ms) | BFS (ms) | União-Busca (ms) |
|---|---|---|---|---|---|
| 10.000 | 5.000 | 5.003 | 1,16 | **0,85** | 3,37 |
| 10.000 | 10.000 | 1.609 | **1,22** | 1,46 | 2,90 |
| 10.000 | 20.000 | 207 | 1,54 | **1,14** | 1,47 |
| 100.000 | 50.000 | 50.001 | 11,90 | 14,57 | **10,20** |
| 100.000 | 100.000 | 16.279 | 21,81 | 12,93 | **7,66** |
| 100.000 | 200.000 | 1.870 | 44,86 | 35,30 | **16,00** |
| 1.000.000 | 500.000 | 500.000 | 260,46 | 213,32 | **105,92** |
| 1.000.000 | 1.000.000 | 161.868 | 387,04 | 313,81 | **109,38** |
| 1.000.000 | 2.000.000 | 19.266 | 521,39 | 386,46 | **264,14** |

*Tabela 4 — Tempos medidos (OpenJDK 21; em verde, o menor de cada linha). Os valores absolutos dependem da máquina.*

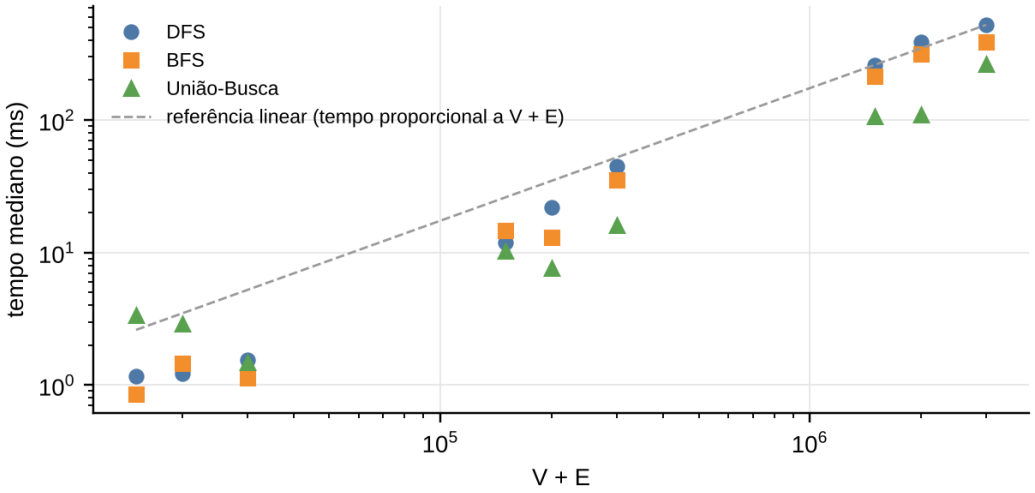



*Figura 3 — Tempo em função de V + E (escala log-log); a reta tracejada indica crescimento linear.*

Três observações. Primeiro, os tempos acompanham a reta de crescimento linear da Figura 3, como prevê a análise. Entre $10^5$ e $10^6$ vértices, porém, o grafo 10 vezes maior levou de 10,4 a 24,3 vezes mais tempo: além do trabalho proporcional a $V + E$, o grafo grande deixa de caber na memória cache do processador, e cada acesso à memória passa a custar mais. Segundo, a partir de $10^5$ vértices a União-Busca foi a mais rápida nesta implementação, apesar de ter a complexidade assintótica ligeiramente pior. A causa é o uso da memória: a DFS e a BFS saltam entre listas de adjacência de vértices espalhados (listas de `Integer`, com um objeto por vizinho), enquanto a União-Busca percorre as arestas em sequência e trabalha só sobre dois vetores compactos de `int`, que aproveitam melhor a memória cache. Uma representação compacta do grafo (vetores de deslocamentos, o formato CSR) reduziria essa diferença. Terceiro, a coluna $k$ ilustra a transição de fase dos grafos aleatórios (Erdős e Rényi, 1960): no ponto crítico $m = 0{,}5n$ (grau médio 1) ainda não há componente grande — o grafo é quase uma floresta, e $k$ fica praticamente igual ao limite inferior $n - m$ da Seção 2.1 —, enquanto com $m = 2n$ já existe um componente gigante que reúne a grande maioria dos vértices.

# 4. Aplicação: municípios isolados por uma cheia

## 4.1 O problema

No período chuvoso, as cheias do rio Jaguaribe e de seus afluentes podem interditar estradas no Baixo Jaguaribe e no Litoral Leste do Ceará. Quando isso acontece, a Defesa Civil precisa responder rapidamente a perguntas como as da Tabela 5 — e todas elas são perguntas sobre componentes conexos.

| Pergunta da Defesa Civil | Tradução em termos de grafos |
|---|---|
| Quais municípios ainda se alcançam por estrada? | Os componentes conexos da rede após a cheia. |
| Algum grupo ficou sem acesso a uma base de apoio? | Algum componente não contém vértice marcado como base? |
| Ainda dá para ir de Fortaleza a Aracati? | c(Fortaleza) = c(Aracati)? — consulta em O(1). |
| Quais estradas são pontos frágeis da rede? | Arestas cuja remoção aumenta o número de componentes. |
| Em que ordem reabrir as estradas? | Arestas que unem componentes distintos (União-Busca incremental). |

*Tabela 5 — Perguntas do problema e sua formulação como problemas de grafos.*

### 4.2 Modelagem

- **Vértices:** os 18 municípios da região estudada (Fortaleza, Aquiraz, Pindoretama, Cascavel, Beberibe, Fortim, Aracati, Icapuí, Pacajus, Chorozinho, Russas, Morada Nova, Limoeiro do Norte, Quixeré, Tabuleiro do Norte, Palhano, Jaguaruana e Itaiçaba).
- **Arestas:** as 21 ligações rodoviárias entre municípios vizinhos. O grafo é **não direcionado**, porque as estradas são de mão dupla, e **não ponderado**, porque a pergunta é se existe caminho, não qual é o mais curto.
- **Atributo dos vértices:** quatro municípios abrigam uma **base de apoio** (hospital ou posto da Defesa Civil): Fortaleza, Aracati, Russas e Limoeiro do Norte.
- **A cheia:** é modelada como a **remoção** das 8 arestas correspondentes às estradas que cruzam a várzea do Jaguaribe. A reabertura de uma estrada é a **inserção** da aresta de volta.

**Observação sobre os dados.** Os municípios são reais e as posições na Figura 4 são aproximadas, mas o conjunto de ligações é um **modelo simplificado e ilustrativo**, construído para o seminário: ele não reproduz a malha viária oficial, e as bases de apoio e as estradas alagadas foram escolhidas para exercitar o algoritmo. A rede fica no arquivo `dados/rede_litoral_leste.txt` e pode ser substituída por dados reais sem nenhuma mudança no código.

### 4.3 A rede em condições normais

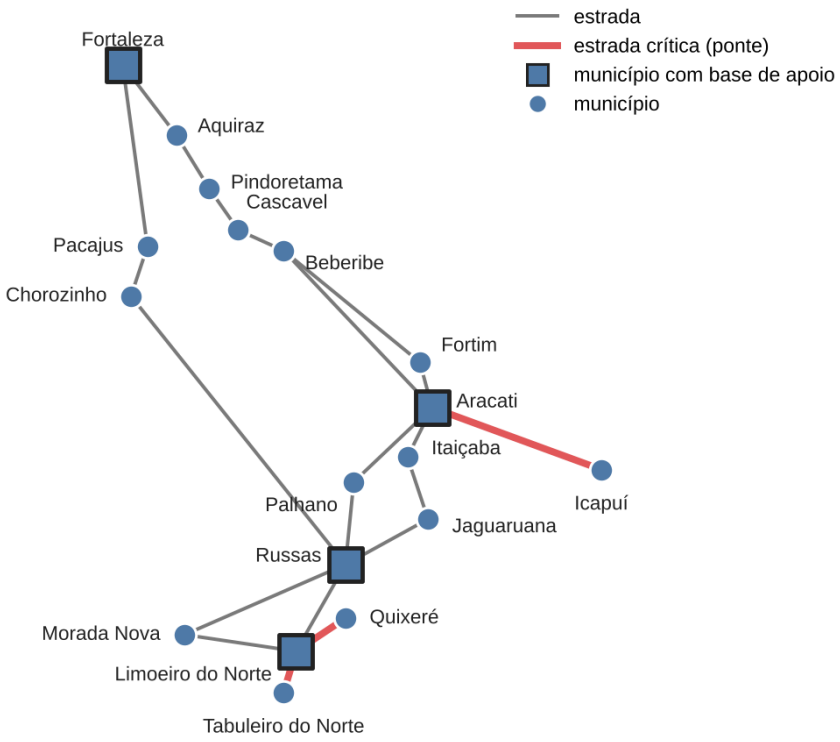



*Figura 4 — Rede modelada: 18 municípios e 21 estradas; em vermelho, as estradas críticas.*

Antes da cheia, a DFS encontra **um único componente**: a rede é conexa. Como primeira análise, o programa testa cada uma das 21 estradas: remove-a de uma cópia do grafo, recalcula os componentes e verifica se o número aumentou. As estradas cuja remoção desconecta a rede são as **pontes** do grafo — pontos frágeis que merecem prioridade de manutenção. São 3:

| Estrada crítica | Município isolado se ela for interditada |
|---|---|
| Aracati -- Icapuí | Icapuí |
| Limoeiro do Norte -- Quixeré | Quixeré |
| Limoeiro do Norte -- Tabuleiro do Norte | Tabuleiro do Norte |

*Tabela 6 — Estradas críticas da rede (saída real do programa).*

```java
for (int[] e : rede.arestas()) {
    Grafo teste = rede.copia();
    teste.removerAresta(e[0], e[1]);
    Componentes c = dfs.calcular(teste);
    if (c.quantidade() > normal.quantidade()) {
        criticas++;
        List<Integer> menor = menorLado(c, e);
        out.printf("    - %-40s isola: %s%n", rotulo(e), nomes(menor));
    }
}
```

O teste é feito com o próprio algoritmo de componentes, repetido $m$ vezes, ao custo de $O(m \cdot (n + m))$ — instantâneo para esta rede. Para redes muito grandes, as pontes podem ser encontradas em $O(n + m)$ por uma única DFS (algoritmo de Tarjan), um refinamento da mesma busca. Das 21 estradas, as outras 18 fazem parte de ciclos e, isoladamente, não desconectam ninguém.

### 4.4 Após a cheia

Removidas as 8 estradas interditadas, o programa recalcula os componentes com a DFS e confere o resultado com a BFS e a União-Busca (os três coincidem). A rede se divide em **5 componentes conexos**:

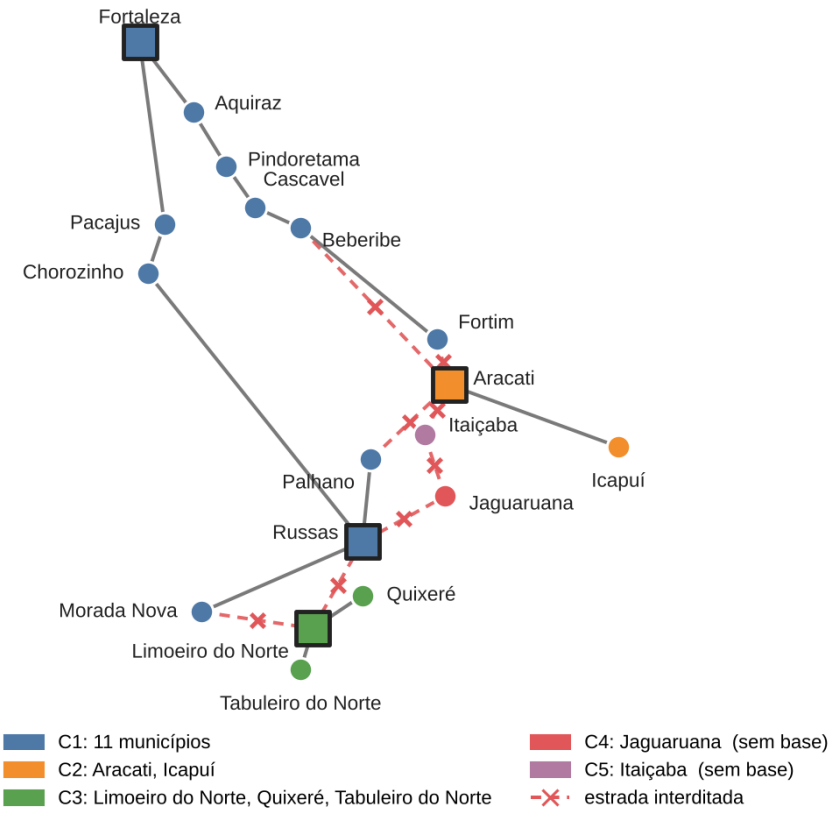



*Figura 5 — Rede após a cheia: cada cor é um componente conexo; tracejado, as estradas interditadas.*

| Componente | Municípios | Situação |
|---|---|---|
| C1 (11) | Fortaleza, Aquiraz, Pindoretama, Cascavel, Beberibe, Fortim, Pacajus, Chorozinho, Russas, Morada Nova, Palhano | atendido por Fortaleza, Russas |
| C2 (2) | Aracati, Icapuí | atendido por Aracati |
| C3 (3) | Limoeiro do Norte, Quixeré, Tabuleiro do Norte | atendido por Limoeiro do Norte |
| C4 (1) | Jaguaruana | **sem base de apoio → socorro aéreo/fluvial** |
| C5 (1) | Itaiçaba | **sem base de apoio → socorro aéreo/fluvial** |

*Tabela 7 — Componentes conexos após a cheia e situação de cada grupo (saída real do programa).*

A resposta à pergunta central da Defesa Civil é imediata: **2 grupos estão sem acesso por terra a nenhuma base de apoio** — Jaguaruana e Itaiçaba ficaram completamente ilhadas, e o socorro a elas precisa ser feito por via aérea ou fluvial. Os demais grupos, embora separados entre si, têm cada um ao menos uma base: o componente C1 é atendido por Fortaleza e Russas; C2, por Aracati; e C3, por Limoeiro do Norte. Note que Aracati e Icapuí continuam ligadas entre si, mas não ao restante da rede.

### 4.5 Consultas de conectividade

Com os rótulos calculados, saber se dois municípios ainda se alcançam custa uma comparação, como mostra a Tabela 8. Uma busca de caminho a cada pergunta custaria $O(n + m)$ por consulta.

| Par de municípios | Antes da cheia | Depois da cheia |
|---|---|---|
| Fortaleza -- Aracati | sim | **não** |
| Fortaleza -- Fortim | sim | sim |
| Icapuí -- Aracati | sim | sim |
| Itaiçaba -- Jaguaruana | sim | **não** |
| Quixeré -- Russas | sim | **não** |

*Tabela 8 — Consultas respondidas em O(1) a partir dos rótulos dos componentes.*

### 4.6 Plano de reparo: União-Busca incremental

Suponha que a Defesa Civil tenha planejado reabrir as estradas na ordem do arquivo de dados. A cada reabertura, recalcular os componentes do zero custaria $O(n + m)$; com a União-Busca, cada estrada reaberta é uma única chamada `unir(u, v)`, e o valor devolvido já diz se ela reconectou dois grupos ou se ligou municípios que já se alcançavam. Junto a cada raiz, o programa guarda se aquele grupo contém uma base de apoio:

```
int ra = uf.buscar(e[0]), rb = uf.buscar(e[1]);
boolean basesUniao = temBase[ra] || temBase[rb];
boolean util = uf.unir(e[0], e[1]);
if (util) {
    uteis++;
    temBase[uf.buscar(e[0])] = basesUniao;
}
```

| Passo | Estrada reaberta | Efeito | Componentes | Grupos sem base |
|---|---|---|---|---|
| 0 | (situação após a cheia) | - | 5 | 2 |
| 1 | Beberibe -- Aracati | reconecta | 4 | 2 |
| 2 | Fortim -- Aracati | redundante | 4 | 2 |
| 3 | Palhano -- Aracati | redundante | 4 | 2 |
| 4 | Russas -- Limoeiro do Norte | reconecta | 3 | 2 |
| 5 | Morada Nova -- Limoeiro do Norte | redundante | 3 | 2 |
| 6 | Itaiçaba -- Aracati | reconecta | 2 | 1 |
| 7 | Jaguaruana -- Itaiçaba | reconecta | 1 | 0 |
| 8 | Russas -- Jaguaruana | redundante | 1 | 0 |

*Tabela 9 — Simulação da ordem de reparo planejada.*

Duas conclusões saem da simulação. A primeira confirma a teoria da Seção 2.1: das 8 estradas, apenas **4 = 5 − 1** reconectam grupos; as outras 4 são **redundantes** para a conectividade — ligam municípios que, naquele momento, já se alcançam por outro caminho. A segunda é um problema do plano: os dois municípios ilhados, justamente os que estão sem apoio, só são alcançados no **passo 7**. Até lá, as equipes gastam 3 passos reabrindo estradas redundantes.

## 4.7 Ordem de reparo sugerida

A própria União-Busca permite construir uma ordem melhor. O programa adota um critério **guloso**: a cada passo, escolhe primeiro uma estrada que ligue um grupo *sem* base a um grupo *com* base; se não houver, uma que una dois grupos ainda separados; as estradas redundantes ficam para o fim.

| Passo | Estrada reaberta | Efeito | Componentes | Grupos sem base |
|---|---|---|---|---|
| 0 | (situação após a cheia) | - | 5 | 2 |
| 1 | Itaiçaba -- Aracati | reconecta | 4 | 1 |
| 2 | Jaguaruana -- Itaiçaba | reconecta | 3 | 0 |
| 3 | Beberibe -- Aracati | reconecta | 2 | 0 |
| 4 | Russas -- Limoeiro do Norte | reconecta | 1 | 0 |
| 5 | Fortim -- Aracati | redundante | 1 | 0 |
| 6 | Palhano -- Aracati | redundante | 1 | 0 |
| 7 | Morada Nova -- Limoeiro do Norte | redundante | 1 | 0 |
| 8 | Russas -- Jaguaruana | redundante | 1 | 0 |

*Tabela 10 — Simulação da ordem de reparo sugerida pelo critério guloso.*

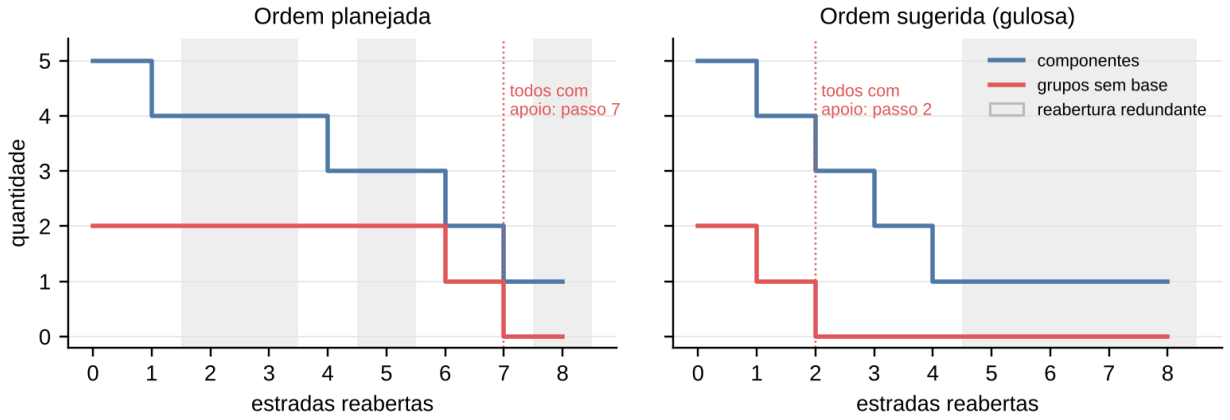



*Figura 6 — Componentes e grupos sem base de apoio ao longo do reparo; as faixas cinzas marcam reaberturas redundantes.*

Com a ordem sugerida, todos os grupos passam a ter acesso a uma base de apoio no **passo 2** (em vez do passo 7) e a rede inteira se reunifica no **passo 4** (em vez do passo 7), usando exatamente as 4 estradas necessárias. Essa escolha é mínima: cada uma das 4 estradas liga dois componentes que ainda estavam separados, e nenhuma forma ciclo com as anteriores — o mesmo teste que o algoritmo de Kruskal faz, também com União-Busca, para decidir se uma aresta entra na árvore geradora mínima.

O critério não leva em conta o custo ou a duração de cada obra, que dados reais trariam. Se cada estrada tivesse um custo de reparo, a pergunta “qual é o conjunto mais barato de estradas que reconecta todos os grupos?” passaria a ser um problema de árvore geradora mínima sobre o grafo dos componentes, resolvido pelo próprio Kruskal.

### 4.8 Limitações e extensões

- O modelo é **ilustrativo**: com a malha viária real (por exemplo, dados abertos do DNIT ou da Superintendência de Obras Públicas do Ceará), basta trocar o arquivo de dados.
- Os componentes dizem **se** é possível chegar, não **quanto custa**. Para escolher a rota mais curta até uma base, o grafo precisaria de pesos (distâncias) e de um algoritmo de caminho mínimo, como Dijkstra.
- Uma cheia evolui com o tempo. Como a União-Busca não trata remoções, uma sequência de interdições é tratada recalculando os componentes com DFS a cada mudança, o que, em $O(n + m)$, é instantâneo para redes desse porte.

## 5. Conclusão

Componentes conexos são a resposta formal à pergunta “quem alcança quem?”. A alcançabilidade é uma relação de equivalência, e por isso divide os vértices em grupos disjuntos que podem ser encontrados em tempo linear, $O(n + m)$, por uma busca em profundidade ou em largura, ou em tempo quase linear pela União-Busca, que tem a vantagem de ser incremental.

Na implementação em Java, as decisões mais importantes foram usar listas de adjacência em vez de matriz, fazer a DFS com pilha explícita — a versão recursiva estoura a pilha da JVM já com 10 mil vértices em caminho —, implementar a União-Busca com união por tamanho e compressão de caminho e normalizar os rótulos para que os três algoritmos sejam diretamente comparáveis. Os 4.073 testes, que incluem a comparação com o fecho transitivo em 500 grafos aleatórios, e as medições confirmam a correção e o crescimento linear do tempo.

Na aplicação, o mesmo algoritmo respondeu a todas as perguntas da Defesa Civil: identificou os 5 grupos formados pela cheia e os 2 municípios ilhados sem apoio, respondeu a consultas de conectividade em tempo constante, apontou as 3 estradas críticas da rede e, com a União-Busca, mostrou que só 4 das 8 estradas interditadas são necessárias para reunificar a rede, propondo uma ordem de reparo que leva apoio aos municípios isolados no passo 2, e não no 7.

## 6. Referências


- CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. *Algoritmos: teoria e prática*. 3. ed. Rio de Janeiro: Elsevier, 2012. Cap. 21 (Estruturas de dados para conjuntos disjuntos) e cap. 22 (Algoritmos elementares em grafos).
- SEDGEWICK, R.; WAYNE, K. *Algorithms*. 4. ed. Boston: Addison-Wesley, 2011. Seções 1.5 (Union-Find) e 4.1 (Undirected Graphs).
- FEOFILOFF, P.; KOHAYAKAWA, Y.; WAKABAYASHI, Y. *Uma introdução sucinta à teoria dos grafos*. São Paulo: IME-USP, 2011.
- BONDY, J. A.; MURTY, U. S. R. *Graph Theory*. London: Springer, 2008. (Graduate Texts in Mathematics, 244).
- HOPCROFT, J.; TARJAN, R. Algorithm 447: efficient algorithms for graph manipulation. *Communications of the ACM*, v. 16, n. 6, p. 372–378, 1973.
- TARJAN, R. E. Efficiency of a good but not linear set union algorithm. *Journal of the ACM*, v. 22, n. 2, p. 215–225, 1975.
- ERDŐS, P.; RÉNYI, A. On the evolution of random graphs. *Publications of the Mathematical Institute of the Hungarian Academy of Sciences*, v. 5, p. 17–61, 1960.

## Apêndice A — Como executar

Requisito: JDK 17 ou superior; não há outras dependências. Na pasta do projeto:

```
# compilar
javac -encoding UTF-8 -d out src/componentes/*.java testes/componentes/*.java

# executar
java -cp out componentes.Main                # aplicação (Seção 4)
java -cp out componentes.Main exemplo        # exemplo didático (Seção 2)
java -cp out componentes.Main desempenho     # medições (Seção 3.8)
java -cp out componentes.Main componentes dados/rede_litoral_leste.txt bfs
java -cp out componentes.TestesComponentes   # testes automatizados
```

O arquivo `README.md` do repositório traz as mesmas instruções, a estrutura de pastas e o formato dos arquivos de grafo (`aresta: A -- B`, `vertice: X` etc.). No Windows, se os acentos aparecerem trocados no terminal, execute `chcp 65001` antes.

## Apêndice B — Saída completa da aplicação

Saída do comando `java -cp out componentes.Main`, da qual foram extraídas as tabelas da Seção 4:

```
================================================================================
 1. REDE EM CONDIÇÕES NORMAIS
================================================================================
  18 municípios, 21 estradas, 1 componente(s) conexo(s)
  Bases de apoio: Fortaleza, Aracati, Russas, Limoeiro do Norte

  Estradas críticas (se apenas ela for interditada, a rede se divide):
    - Aracati -- Icapuí                       isola: Icapuí
    - Limoeiro do Norte -- Quixeré            isola: Quixeré
    - Limoeiro do Norte -- Tabuleiro do Norte  isola: Tabuleiro do Norte
  3 de 21 estradas são críticas.

================================================================================
 2. APÓS A CHEIA
================================================================================
  Estradas interditadas:
    x Beberibe -- Aracati
    x Fortim -- Aracati
    x Palhano -- Aracati
    x Russas -- Limoeiro do Norte
    x Morada Nova -- Limoeiro do Norte
    x Itaiçaba -- Aracati
    x Jaguaruana -- Itaiçaba
    x Russas -- Jaguaruana
  Conferência: DFS = BFS = União-Busca (mesmos componentes)

  Resultado: 5 componentes conexos (antes: 1)

  Componente 1 (11 municípios): Fortaleza, Aquiraz, Pindoretama, Cascavel, Beberibe, Fortim, Pacajus,
      Chorozinho, Russas, Morada Nova, Palhano
     situação: atendido por Fortaleza, Russas
  Componente 2 (2 municípios): Aracati, Icapuí
     situação: atendido por Aracati
  Componente 3 (3 municípios): Limoeiro do Norte, Quixeré, Tabuleiro do Norte
     situação: atendido por Limoeiro do Norte
  Componente 4 (1 município): Jaguaruana
     situação: SEM BASE DE APOIO -> socorro aéreo/fluvial
  Componente 5 (1 município): Itaiçaba
     situação: SEM BASE DE APOIO -> socorro aéreo/fluvial

  2 componente(s) sem base de apoio.

================================================================================
 3. CONSULTAS DE CONECTIVIDADE (O(1) após rotular)
================================================================================
  Fortaleza -- Aracati                    antes: sim | depois: não
  Fortaleza -- Fortim                     antes: sim | depois: sim
  Icapuí -- Aracati                       antes: sim | depois: sim
  Itaiçaba -- Jaguaruana                  antes: sim | depois: não
  Quixeré -- Russas                       antes: sim | depois: não

================================================================================
 4. PLANO DE REPARO DA DEFESA CIVIL (simulado com União-Busca)
================================================================================
  Pas. Estrada reaberta                  Efeito       Componentes  Sem base
  0    (situação após a cheia)           -            5            2
```

```
  1    Beberibe -- Aracati               reconecta    4            2
  2    Fortim -- Aracati                 redundante   4            2
  3    Palhano -- Aracati                redundante   4            2
  4    Russas -- Limoeiro do Norte       reconecta    3            2
  5    Morada Nova -- Limoeiro do Norte  redundante   3            2
  6    Itaiçaba -- Aracati               reconecta    2            1
  7    Jaguaruana -- Itaiçaba            reconecta    1            0
  8    Russas -- Jaguaruana              redundante   1            0

  Ordem planejada: todos os grupos com acesso a uma base no passo 7; rede reunificada no passo 7.
  Estradas que efetivamente reconectaram grupos: 4 de 8.

================================================================================
 5. ORDEM DE REPARO SUGERIDA
================================================================================
  Critério guloso: primeiro as estradas que ligam um grupo SEM base a um grupo COM base;
  depois, as que unem dois grupos ainda separados; estradas redundantes ficam por último.

  Pas. Estrada reaberta                  Efeito       Componentes  Sem base
  0    (situação após a cheia)           -            5            2
  1    Itaiçaba -- Aracati               reconecta    4            1
  2    Jaguaruana -- Itaiçaba            reconecta    3            0
  3    Beberibe -- Aracati               reconecta    2            0
  4    Russas -- Limoeiro do Norte       reconecta    1            0
  5    Fortim -- Aracati                 redundante   1            0
  6    Palhano -- Aracati                redundante   1            0
  7    Morada Nova -- Limoeiro do Norte  redundante   1            0
  8    Russas -- Jaguaruana              redundante   1            0

  Ordem sugerida: todos os grupos com acesso a uma base no passo 2; rede reunificada no passo 4.
  Estradas que efetivamente reconectaram grupos: 4 de 8.
```